COMP2025 Assignment: Weather in Parramatta

COMP2025 Introduction to Data Science

Assignment, Spring 2026

Weather in Parramatta, NSW

Name: FULL NAME
Student number: STUDENT NUMBER

Declaration

By including this statement, we the authors of this work, verify that:

- I hold a copy of this assignment that we can produce if the original is lost or damaged.
- I hereby certify that no part of this assignment/product has been copied from any other student's work or from any other source (including generative sources) except where due acknowledgement is made in the assignment.
- No part of this assignment/product has been written/produced for us by another person except where such collaboration has been authorised by the subject lecturer/tutor concerned.
- I am aware that this work may be reproduced and submitted to plagiarism detection software programs for the purpose of detecting possible plagiarism (which may retain a copy on its database for future plagiarism checking).
- I hereby certify that we have read and understand what the School of Computer, Data and Mathematical Science defines as minor and substantial breaches of misconduct as outlined in the Subject Outline for this unit.

Acknowledgement: In preparing this assignment I drew on general resources available online, including R documentation and package vignettes, statistics tutorials, and discussion forums such as Stack Overflow, for guidance on methods and to help debug my code. Generative AI tools were also used as a study and coding aid to help plan the analysis and explain statistical methods. All code was written and run by me, and all results come from my own downloaded data.

Part 1: Gathering and Exploring the Data

Aim

The aim of this part is to download at least three full years of daily weather data for Parramatta from the Open-Meteo Historical Weather API, to describe each variable and its units, and to explore the data graphically so that the later parts have a sound basis for choosing predictors and models.

Location

I chose Parramatta, in Greater Western Sydney (latitude -33.8150, longitude 151.0011), because it is in the part of Sydney where I live and study, so it is the weather I actually experience. It is also a good location for this project. Parramatta is inland, so summers there are typically hotter than on the coast, it has four clear seasons, and it has a good mix of dry and rainy days. The data covers five full years, 1 January 2021 to 31 December 2025 (1826 daily observations), downloaded once from the Open-Meteo Historical Weather API using the `httr` and `jsonlite` packages and saved to a CSV file. There were no missing values, so no cleaning was needed.

Downloading the data

The data was downloaded once from the Open-Meteo Historical Weather API with the code below (shown but not re-run when the report is built) and saved to `parramatta_weather.csv`.

```
library(httr)
library(jsonlite)

# Parramatta coordinates
lat <- -33.8150
lon <- 151.0011

# Daily variables to download
daily_vars <- c("temperature_2m_max", "temperature_2m_min", "precipitation_sum",
                "wind_speed_10m_max", "shortwave_radiation_sum", "sunshine_duration")

# Request five full years of daily data
resp <- GET(
  "https://archive-api.open-meteo.com/v1/archive",
  query = list(latitude = lat, longitude = lon,
               start_date = "2021-01-01", end_date = "2025-12-31",
               daily = paste(daily_vars, collapse = ","),
               timezone = "Australia/Sydney"))
stop_for_status(resp)

raw <- fromJSON(content(resp, as = "text", encoding = "UTF-8"))
weather <- as.data.frame(raw$daily)
write.csv(weather, "parramatta_weather.csv", row.names = FALSE)
```

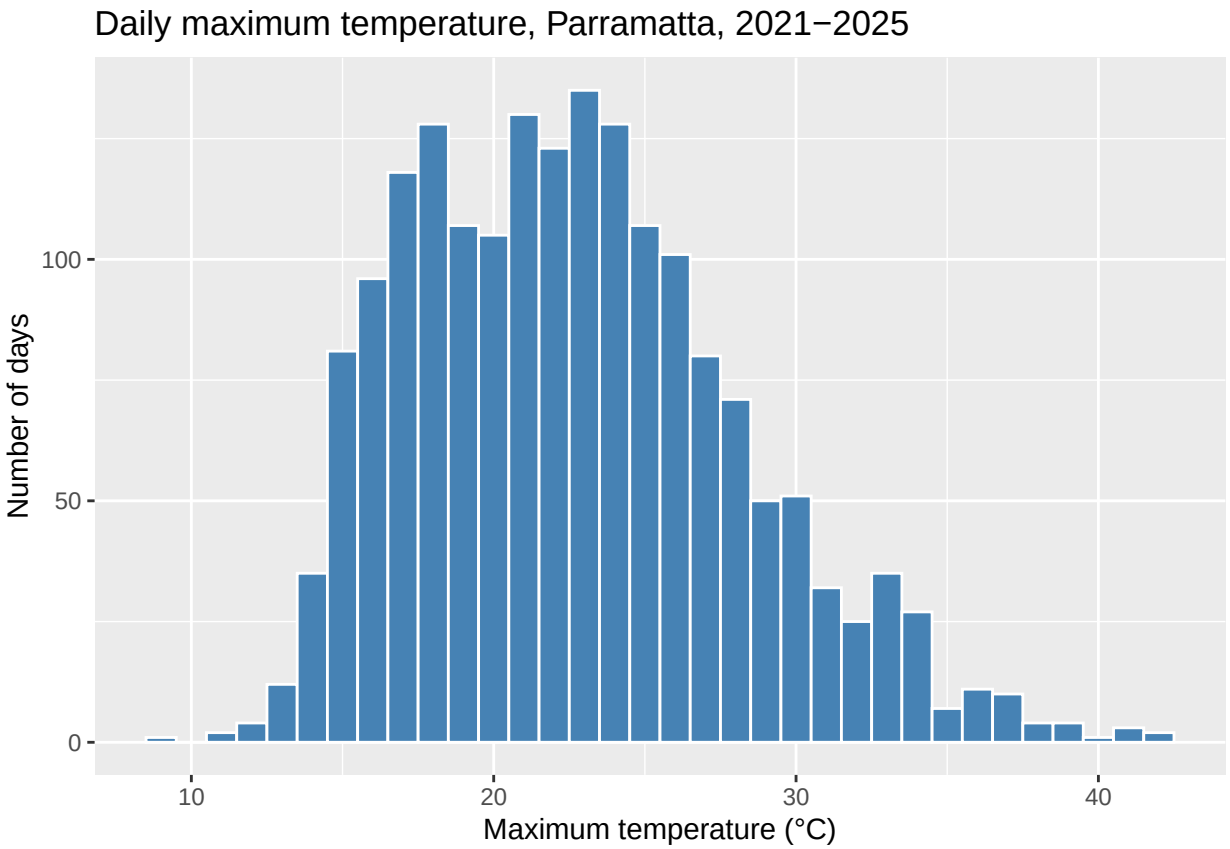
Variables

Variable	Description	Unit
temperature_2m_max	Maximum air temperature during the day, measured 2 m above ground	°C
temperature_2m_min	Minimum air temperature during the day, measured 2 m above ground	°C
precipitation_sum	Total daily precipitation (rain, showers and snow water equivalent)	mm
wind_speed_10m_max	Maximum wind speed during the day, measured 10 m above ground	km/h
shortwave_radiation_sum	Total solar (shortwave) radiation reaching the ground over the day	MJ/m²
sunshine_duration	Time with direct sunshine during the day; converted to hours (SunHours) for readability	seconds, then hours

Two further variables were derived from the date: day of year (1 to 366) and a categorical season (Summer = Dec to Feb, Autumn = Mar to May, Winter = Jun to Aug, Spring = Sep to Nov), following the southern hemisphere calendar.

Histogram of daily maximum temperature

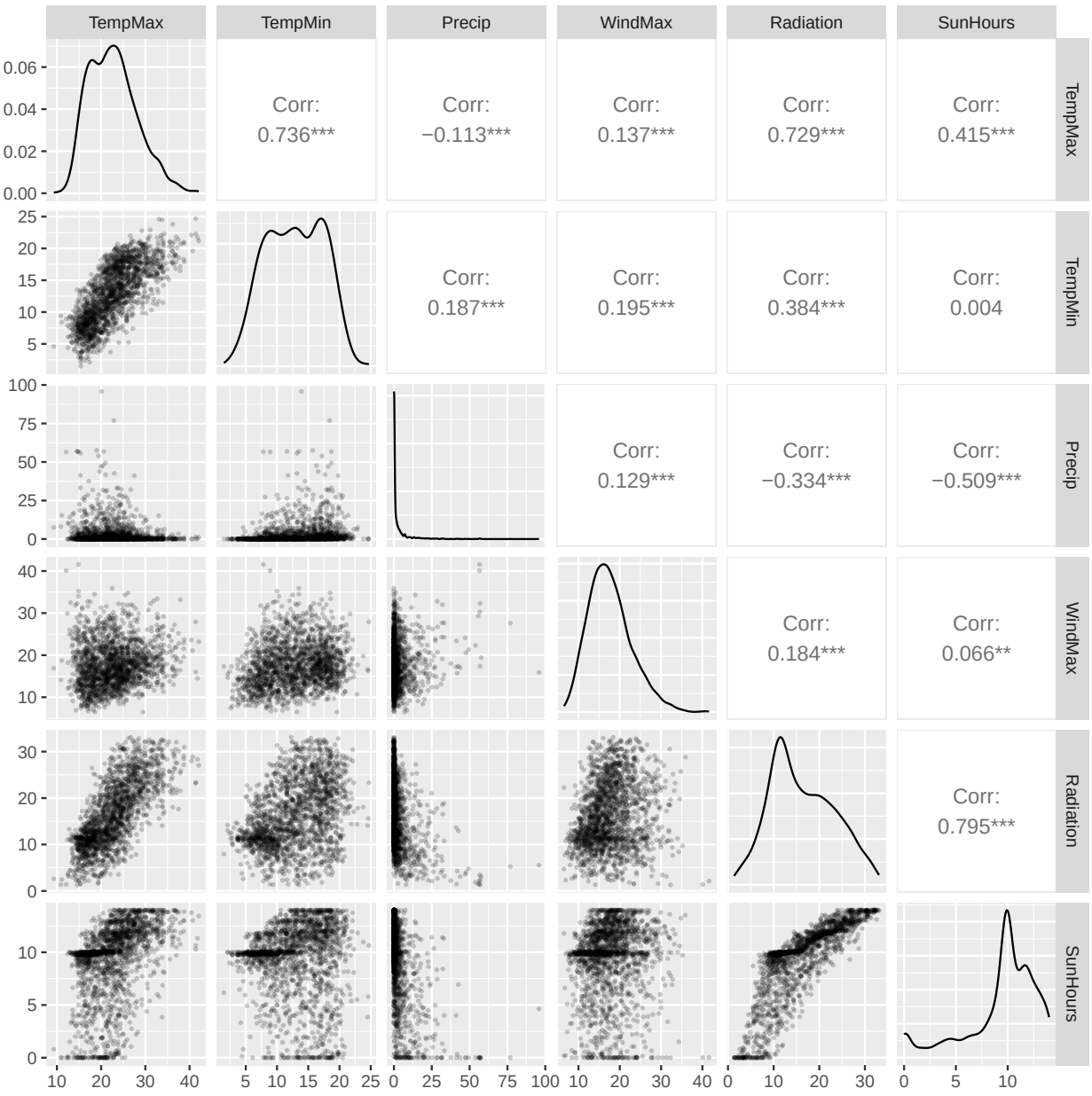
```
ggplot(weather, aes(x = temperature_2m_max)) +
  geom_histogram(binwidth = 1, fill = "steelblue", colour = "white") +
  labs(title = "Daily maximum temperature, Parramatta, 2021-2025",
       x = "Maximum temperature (°C)", y = "Number of days")
```



The daily maximum temperature has a mean of 22.7 °C and a median of 22.3 °C, with values from 9.3 °C to 42.1 °C. The distribution is centred around 18 to 25 °C and is not perfectly bell-shaped. It is flat-topped, with small peaks near 18 °C and 23 °C, which is what we would expect from a mixture of cooler (autumn and winter) and warmer (spring and summer) days. It is also right-skewed (sample skewness 0.54). There is a sharp lower cut-off near 14 °C, so Parramatta rarely has cold days, and a long upper tail of hot days. There were 39 days of 35 °C or more, which are most likely summer heatwave days and are typical of inland Western Sydney.

Scatterplot matrix

```
ggpairs(num_vars, lower = list(continuous = wrap("points", alpha = 0.2, size = 0.5)))
```



Commentary

Relationships. TempMax is strongly positively related to TempMin (r = 0.74) and to Radiation (r = 0.73). Warm days tend to be followed by warm nights, and days with more incoming solar energy are hotter. TempMax is only moderately related to SunHours (r = 0.41) and very weakly related to WindMax (r = 0.14) and Precip (r = -0.11). Radiation and SunHours are strongly correlated with each other (r = 0.8) because both measure how sunny the day was. Including both in a regression would cause multicollinearity, which inflates standard errors and makes individual coefficients hard to interpret. Precip is negatively correlated with SunHours (r = -0.51) and Radiation (r = -0.33), as expected, because cloudy, wet days receive less sun. TempMin and SunHours are essentially uncorrelated (r = 0.004), which suggests overnight minimums do not depend on how sunny the previous day was. With 1826 observations even small correlations are statistically significant, so I focus on the size of r rather than the p-values.

Seasonality. TempMin, TempMax and Radiation all follow the seasons, which is why they move together.

The multimodal shapes of TempMin and Radiation (flat or lumpy densities) reflect the summer and winter halves of the year. This means a straight-line relationship between day of year and temperature would fit poorly, which motivates the non-linear models in Part 7.

Skew and unusual observations. Precip is extremely right-skewed. 41% of days recorded exactly 0 mm, the median is 0.2 mm, the mean is 2.89 mm, and the maximum is 95.8 mm in a single day. WindMax is mildly right-skewed. SunHours has a hard ceiling at 14 hours, which is the physical limit of daylight, a pile-up of days near 0 hours (heavily overcast days) and an unexplained horizontal band of days at exactly 10 hours. I do not know the cause of the 10-hour band, but it may be an artefact of how the sunshine duration is derived rather than real weather. The hottest days (up to 42.1 °C) and the very wet days are genuine extreme events and I keep them in the data.

Implications for modelling. (1) For Part 2, TempMin, Radiation and season are promising predictors of TempMax, but I will use only one of Radiation and SunHours to avoid multicollinearity. (2) Precip is too skewed to be a sensible response in linear regression, so in Part 3 I convert it to a binary rain-day indicator. (3) Because the sunshine and rain variables are linked, the rain-day classifiers should not use predictors used to define the indicator. (4) Seasonality means non-linear terms in day of year are needed (Part 7).

Conclusion for Parramatta. Parramatta's weather over 2021 to 2025 shows clear structure that the later parts can exploit. Maximum and minimum temperature and solar radiation rise and fall together with the seasons, so a seasonal or non-linear term will be needed; precipitation is far too skewed to model directly, which is why it becomes a binary rain-day response; and radiation and sunshine hours carry almost the same information, so only one of them should be used. These observations set up the predictor choices in Parts 2, 3 and 7.

Limitations. Only five years of data are used, so rare extremes (such as a severe heatwave) are under-represented. The values are from a gridded reanalysis-based model rather than a single weather station, so they may differ from official Parramatta station readings. Consecutive days are also not independent (weather is autocorrelated), which means the usual statistical assumptions are only approximately true. A further step would be to compare this with an official Bureau of Meteorology station record.

Part 2: Regression

Aim

The aim of this part is to predict the daily maximum temperature in Parramatta (`temperature_2m_max`) from other weather variables, and to work out which of those variables matter once the others are accounted for. The predictors are minimum temperature, solar radiation, maximum wind speed and season. I leave out sunshine hours because it is strongly correlated with solar radiation (r = 0.8 in Part 1), and I leave out precipitation because it is extremely skewed.

Initial multiple linear regression model

```
# Model 1: TempMax predicted from TempMin, Radiation, WindMax and season
# (season is a factor, so R uses Summer as the baseline level)
m1 <- lm(temperature_2m_max ~ temperature_2m_min + shortwave_radiation_sum +
           wind_speed_10m_max + season, data = weather)

# Coefficients, standard errors, t-tests, R-squared
summary(m1)
```

```
##
## Call:
## lm(formula = temperature_2m_max ~ temperature_2m_min + shortwave_radiation_sum +
##     wind_speed_10m_max + season, data = weather)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -7.2665 -1.6276 -0.2983  1.3270 10.8888
##
## Coefficients:
##                          Estimate Std. Error t value Pr(>|t|)
## (Intercept)               6.28426    0.45335  13.862  < 2e-16 ***
## temperature_2m_min        0.71778    0.01997  35.942  < 2e-16 ***
## shortwave_radiation_sum   0.45024    0.01090  41.312  < 2e-16 ***
## wind_speed_10m_max       -0.06968    0.01200  -5.809 7.40e-09 ***
## seasonAutumn              1.63717    0.20528   7.975 2.66e-15 ***
## seasonWinter              1.69935    0.28120   6.043 1.83e-09 ***
## seasonSpring              0.98298    0.19670   4.997 6.37e-07 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 2.508 on 1819 degrees of freedom
## Multiple R-squared:  0.7861, Adjusted R-squared:  0.7854
## F-statistic:  1114 on 6 and 1819 DF,  p-value: < 2.2e-16
```

```
# 95% confidence intervals for the coefficients
confint(m1)
```

```
##                               2.5 %     97.5 %
## (Intercept)              5.39512317  7.1733891
```

```
## temperature_2m_min       0.67861188  0.7569459
## shortwave_radiation_sum  0.42886177  0.4716117
## wind_speed_10m_max      -0.09320751 -0.0461554
## seasonAutumn             1.23455604  2.0397792
## seasonWinter             1.14783821  2.2508660
## seasonSpring             0.59718860  1.3687635
```

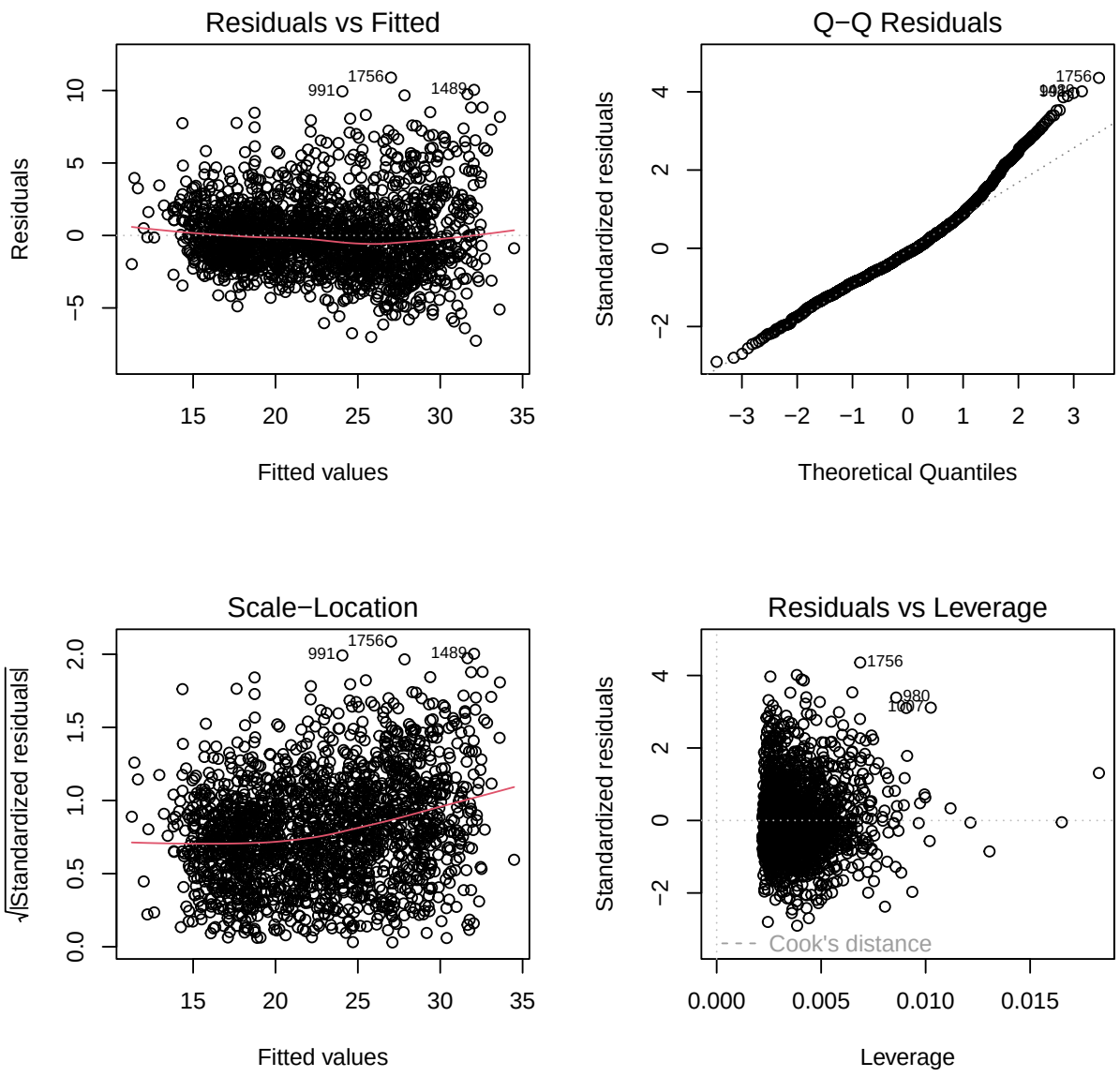
```r
# Variance inflation factors (VIF) for the numeric predictors.
# VIF = 1 / (1 - R^2) from regressing each predictor on the other predictors.
# Values above about 5 would indicate a multicollinearity problem.
xs <- weather[, c("temperature_2m_min", "shortwave_radiation_sum", "wind_speed_10m_max")]
sapply(names(xs), function(v) {
  r2 <- summary(lm(reformulate(setdiff(names(xs), v), v), data = xs))$r.squared
  1 / (1 - r2)
})
```

```
##      temperature_2m_min shortwave_radiation_sum      wind_speed_10m_max
##                1.195905                1.190709                1.055026
```

Interpretation. Holding the other predictors fixed, each extra 1 °C of minimum temperature is associated with a 0.72 °C higher maximum temperature, and each extra 1 MJ/m² of solar radiation with a 0.45 °C higher maximum. Each extra 1 km/h of maximum wind speed is associated with a 0.07 °C lower maximum, which is plausible because windy days tend to mix in cooler air. All three numeric predictors are highly statistically significant ($p < 0.001$). The season coefficients are differences from Summer for a day with the same minimum temperature, radiation and wind. They are positive (Autumn 1.64, Winter 1.7, Spring 0.98 °C) and all are significant. This does not mean autumn is hotter than summer. TempMin and Radiation already carry most of the seasonal signal, so the season terms only describe what is left over once they are accounted for. The VIFs are all below 1.2, so multicollinearity is not a problem, which confirms that leaving out SunHours was a good choice. Model 1 has an R-squared of 0.786 and a residual standard error of 2.51 °C.

Diagnostic plots

```r
# Residuals vs fitted, normal Q-Q, scale-location and residuals vs
# leverage plots for Model 1
par(mfrow = c(2, 2))
plot(m1)
```




```
par(mfrow = c(1, 1))
```

```
# Numbers to support the diagnostic plots
res1 <- resid(m1)

# Skewness and kurtosis of residuals (normal distribution: skewness 0, kurtosis 3)
res_skew <- mean((res1 - mean(res1))^3) / sd(res1)^3
res_kurt <- mean((res1 - mean(res1))^4) / sd(res1)^4
c(skewness = res_skew, kurtosis = res_kurt)
```

```
##  skewness  kurtosis
## 0.7417019 4.1597713
```

```r
# Residual spread (standard deviation) in each season
tapply(res1, weather$season, sd)
```

```
##   Summer   Autumn   Winter   Spring
## 2.871311 1.941805 1.921961 3.079860
```

```r
# Correlation between each day's residual and the previous day's residual
cor(res1[-1], res1[-length(res1)])
```

```
## [1] 0.2715909
```

```r
# Number of days with standardised residuals bigger than 3 in size
sum(abs(rstandard(m1)) > 3)
```

```
## [1] 17
```

Interpretation. The residuals vs fitted plot shows no strong curve, so a linear relationship is a reasonable first approximation for these predictors. The assumptions are not perfectly met though.

1. **Non-normal residuals.** The Q-Q plot bends upwards at the right end. The residuals are right-skewed (skewness 0.74) and heavy-tailed (kurtosis 4.16), and the largest standardised residual is 4.4. These come from hot days that the model under-predicts.
2. **Non-constant variance.** The scale-location plot rises to the right, and the residual standard deviation is 2.9 °C in Summer and 3.1 °C in Spring, but only 1.9 °C in Autumn and 1.9 °C in Winter. Hot-season temperatures are harder to predict because they include sudden heat spikes.
3. **Non-independent errors.** The correlation between consecutive days' residuals is 0.27, so errors are positively autocorrelated, which breaks the independence assumption. The reported standard errors are therefore probably a little too small and the p-values a little too optimistic.
4. **Outliers and leverage.** 17 days have standardised residuals above 3 in size, but no point has high leverage and Cook's distance contours do not appear in the leverage plot, so no single day is driving the fit.

Interaction and transformation

```r
# Model 2: add an interaction between TempMin and Radiation
m2 <- update(m1, . ~ . + temperature_2m_min:shortwave_radiation_sum)

# Model 3: log transform of Radiation
m3 <- update(m1, . ~ . - shortwave_radiation_sum + log(shortwave_radiation_sum))

# Model 4: Radiation effect allowed to differ by season
m4 <- update(m1, . ~ . + shortwave_radiation_sum:season)

# Compare the four models
models <- list(M1_base = m1, M2_TempMin_x_Rad = m2, M3_logRad = m3, M4_Rad_x_season = m4)
data.frame(
  model       = names(models),
  adj_R2      = sapply(models, function(m) summary(m)$adj.r.squared),
  AIC         = sapply(models, AIC),
  residual_se = sapply(models, sigma),
  row.names   = NULL
)
```

```
##              model    adj_R2      AIC residual_se
## 1          M1_base 0.7854301 8549.075    2.508112
## 2 M2_TempMin_x_Rad 0.7876089 8531.435    2.495346
## 3        M3_logRad 0.7604454 8750.201    2.650116
## 4  M4_Rad_x_season 0.7855585 8550.968    2.507361
```

```
# F-tests for the nested models (M1 vs M2, and M1 vs M4)
anova(m1, m2)
```

```
## Analysis of Variance Table
##
## Model 1: temperature_2m_max ~ temperature_2m_min + shortwave_radiation_sum +
##     wind_speed_10m_max + season
## Model 2: temperature_2m_max ~ temperature_2m_min + shortwave_radiation_sum +
##     wind_speed_10m_max + season + temperature_2m_min:shortwave_radiation_sum
##   Res.Df   RSS Df Sum of Sq     F    Pr(>F)
## 1   1819 11443
## 2   1818 11320  1    122.42 19.66 9.807e-06 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
anova(m1, m4)
```

```
## Analysis of Variance Table
##
## Model 1: temperature_2m_max ~ temperature_2m_min + shortwave_radiation_sum +
##     wind_speed_10m_max + season
## Model 2: temperature_2m_max ~ temperature_2m_min + shortwave_radiation_sum +
##     wind_speed_10m_max + season + shortwave_radiation_sum:season
##   Res.Df   RSS Df Sum of Sq     F Pr(>F)
## 1   1819 11443
## 2   1816 11417  3    25.708 1.363 0.2524
```

Choosing the final model. The log transform of Radiation (Model 3) makes the fit clearly worse (AIC 8750.2 against 8549.1 for Model 1), so I reject it. Letting the radiation effect differ by season (Model 4) does not help either. The F-test gives p = 0.25 and its AIC is slightly higher than Model 1. The TempMin x Radiation interaction (Model 2) is the only change that helps. The F-test is significant ($p = 9.8 \times 10^{-6}$), AIC falls from 8549.1 to 8531.4, and adjusted R-squared rises from 0.7854 to 0.7876. **I choose Model 2 as my final model.** The improvement is real but small, so the interaction is a refinement and not a big change in how well we predict.

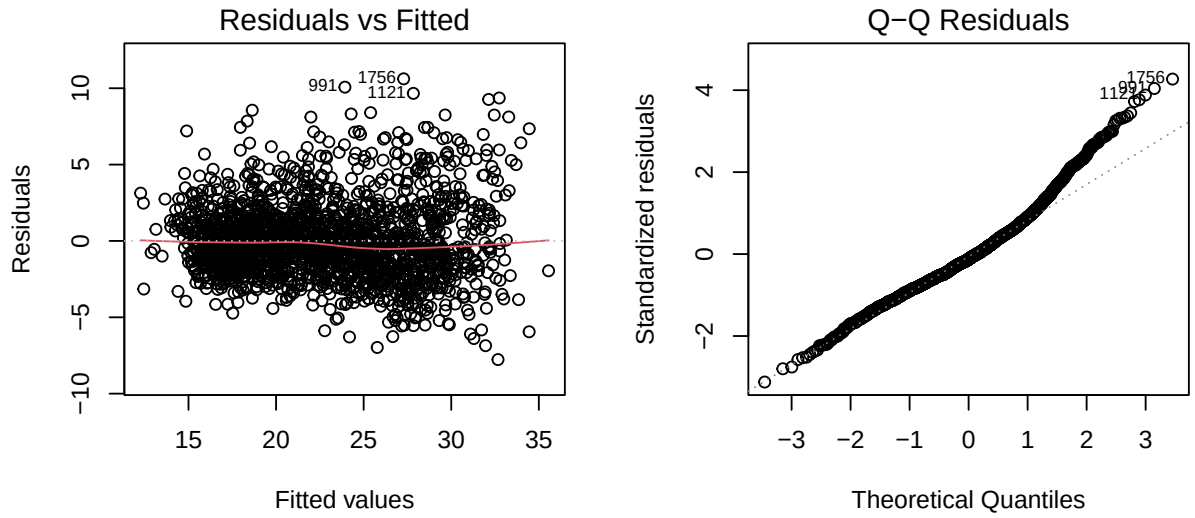
```
# Final model summary
summary(m2)
```

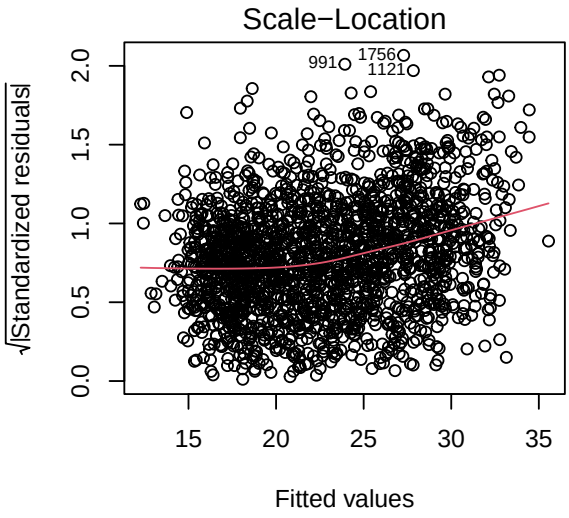
```
##
## Call:
## lm(formula = temperature_2m_max ~ temperature_2m_min + shortwave_radiation_sum +
##     wind_speed_10m_max + season + temperature_2m_min:shortwave_radiation_sum,
##     data = weather)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
```

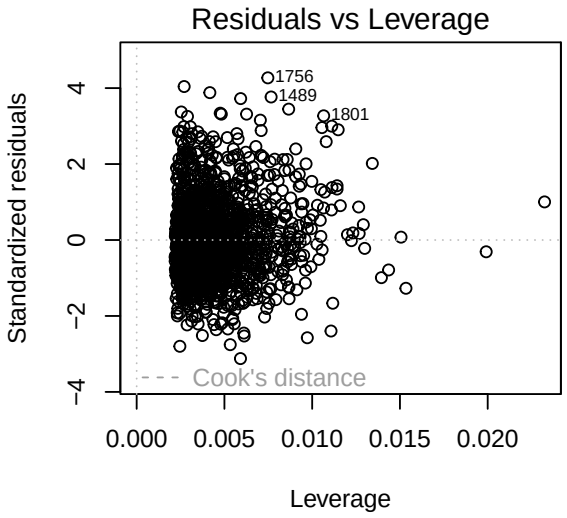
```
## -7.7692 -1.6305 -0.3023  1.3266 10.6167
##
## Coefficients:
##                                            Estimate Std. Error t value
## (Intercept)                                8.839209   0.731762  12.079
## temperature_2m_min                         0.532353   0.046300  11.498
## shortwave_radiation_sum                    0.280121   0.039870   7.026
## wind_speed_10m_max                        -0.066880   0.011951  -5.596
## seasonAutumn                               1.702991   0.204775   8.316
## seasonWinter                               1.424278   0.286567   4.970
## seasonSpring                               1.196874   0.201560   5.938
## temperature_2m_min:shortwave_radiation_sum  0.011507   0.002595   4.434
##                                           Pr(>|t|)
## (Intercept)                                < 2e-16 ***
## temperature_2m_min                         < 2e-16 ***
## shortwave_radiation_sum                   2.99e-12 ***
## wind_speed_10m_max                        2.53e-08 ***
## seasonAutumn                               < 2e-16 ***
## seasonWinter                              7.32e-07 ***
## seasonSpring                              3.45e-09 ***
## temperature_2m_min:shortwave_radiation_sum 9.81e-06 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 2.495 on 1818 degrees of freedom
## Multiple R-squared:  0.7884, Adjusted R-squared:  0.7876
## F-statistic: 967.8 on 7 and 1818 DF,  p-value: < 2.2e-16
```

The interaction coefficient is positive (0.0115), so the effect of solar radiation on the maximum temperature is larger when the night before was warmer. At the average minimum temperature of 12.9 °C, one extra MJ/m² of radiation is associated with about 0.43 °C higher maximum temperature. All terms in the final model remain significant at the 0.1% level. A re-check of the diagnostics (below) shows the same patterns as Model 1.

```r
# Diagnostic plots for the final model (Model 2)
par(mfrow = c(2, 2))
plot(m2)
```








```
par(mfrow = c(1, 1))

# Average residual on hot days (35 C or more): positive means the model under-predicts
mean(resid(m2)[weather$temperature_2m_max >= 35])
```

```
## [1] 6.168576
```

R-squared and conclusion for Parramatta

The final model has an R-squared of 0.788 (adjusted 0.788). This means the model explains about 79% of the day-to-day variation in Parramatta's maximum temperature over 2021 to 2025. The residual standard error is 2.5 °C, so a typical prediction is about 2.5 °C away from the actual value, and roughly 95% of

predictions fall within about ±5 °C. Warm nights and sunny days go with hot days in Parramatta, and wind pulls the maximum down slightly. The model is much worse on the hottest days: on days of 35 °C or more it under-predicts by 6.2 °C on average, so it would understate Parramatta's heatwave peaks.

Limitations and further analysis. (1) R-squared is calculated on the same data used to fit the model, so it is optimistic. Parts 5 and 6 give a more honest error estimate. (2) The residuals are skewed, have unequal variance and are autocorrelated, so the p-values should be read as approximate. Robust standard errors, or a model that includes the previous day's temperature, would deal with this. (3) TempMin and Radiation are measured on the same day as the response, so this model describes the weather rather than forecasting it. (4) The heat spikes suggest that predictors such as wind direction, which are not in this dataset, would help. (5) The model is linear in day-to-day variables, and Part 7 looks at the non-linear seasonal pattern directly.

Part 3: Classification

Aim

The aim is to predict whether a day in Parramatta is a rain day from the other weather measurements. A **rain day** is a day with more than 1 mm of precipitation. I use four predictors: maximum temperature, minimum temperature, maximum wind speed and solar radiation. None of them is used to define a rain day. Precipitation and sunshine hours are left out, because precipitation defines the response and sunshine hours is almost the same measurement as radiation (Part 1). Five classifiers are fitted and compared: logistic regression, LDA, QDA, naive Bayes and KNN.

Creating the response and splitting the data

```
# Packages for the classifiers
library(MASS)    # lda(), qda()
library(class)   # knn(), knn.cv()
library(e1071)   # naiveBayes()

# Binary response: "Rain" if more than 1 mm of precipitation, otherwise "Dry"
weather$rain <- factor(ifelse(weather$precipitation_sum > 1, "Rain", "Dry"),
                       levels = c("Dry", "Rain"))
table(weather$rain)
```

```
##
##  Dry Rain
## 1198  628
```

```
round(prop.table(table(weather$rain)), 3)
```

```
##
##   Dry  Rain
## 0.656 0.344
```

```
# Time-based split: train on 2021-2024, hold out 2025 as the test set.
# Weather is autocorrelated, so a random split would leak information between
# neighbouring days. Predicting a whole later year is a more honest test.
weather$year <- as.numeric(format(weather$date, "%Y"))
train <- weather[weather$year <= 2024, ]
test  <- weather[weather$year == 2025, ]
c(train_days = nrow(train), test_days = nrow(test))
```

```
## train_days  test_days
##       1461        365
```

```
table(train$rain)
```

```
##
##  Dry Rain
##  959  502
```

```r
table(test$rain)
```

```
##
##  Dry Rain
##  239  126
```

```r
# Model formula used by every classifier
f <- rain ~ temperature_2m_max + temperature_2m_min +
            wind_speed_10m_max + shortwave_radiation_sum
```

34% of the 1826 days are rain days, so the classes are imbalanced but both are well represented. A classifier that always said "Dry" would be right 65.5% of the time on the 2025 test set. This is the baseline that any useful classifier has to beat. The data is split by time: 1461 days (2021 to 2024) to fit the models and 365 days (2025) to test them.

Fitting the classifiers

```r
# 1. Logistic regression
glm1 <- glm(f, data = train, family = binomial)
summary(glm1)
```

```
##
## Call:
## glm(formula = f, family = binomial, data = train)
##
## Coefficients:
##                            Estimate Std. Error z value Pr(>|z|)
## (Intercept)                0.7592032  0.4488587   1.691   0.0908 .
## temperature_2m_max        -0.2762056  0.0338383  -8.163 3.28e-16 ***
## temperature_2m_min         0.5375778  0.0336141  15.993  < 2e-16 ***
## wind_speed_10m_max        -0.0002449  0.0139808  -0.018   0.9860
## shortwave_radiation_sum -0.1471749  0.0181138  -8.125 4.47e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 1880  on 1460  degrees of freedom
## Residual deviance: 1204  on 1456  degrees of freedom
## AIC: 1214
##
## Number of Fisher Scoring iterations: 5
```

```r
# 2. LDA (linear boundary, one shared covariance matrix)
lda1 <- lda(f, data = train)
lda1
```

```
## Call:
## lda(f, data = train)
```

```
##
## Prior probabilities of groups:
##       Dry      Rain
## 0.6563997 0.3436003
##
## Group means:
##      temperature_2m_max temperature_2m_min wind_speed_10m_max
## Dry            22.86455           11.63597           17.75714
## Rain           21.81016           14.66733           18.12550
##      shortwave_radiation_sum
## Dry                 17.89282
## Rain                12.90424
##
## Coefficients of linear discriminants:
##                                   LD1
## temperature_2m_max      -0.152078022
## temperature_2m_min       0.330082755
## wind_speed_10m_max       0.002786237
## shortwave_radiation_sum -0.099479918
```

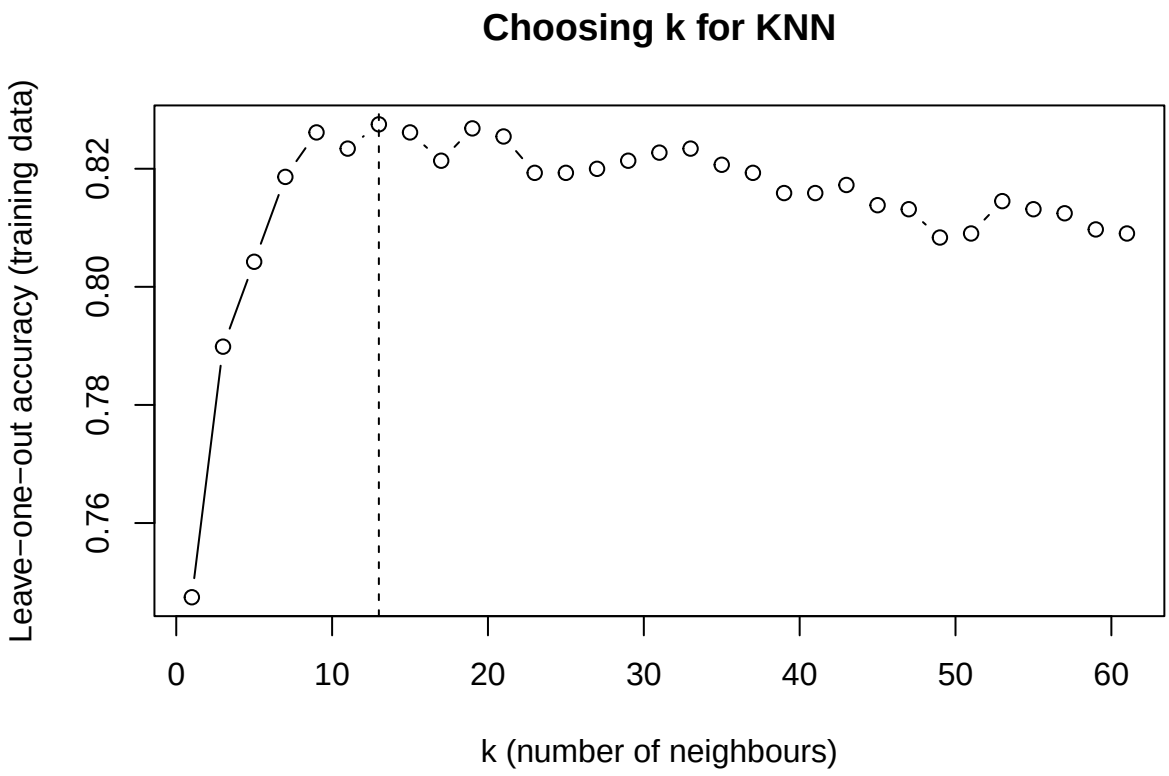
```
# 3. QDA (quadratic boundary, separate covariance matrix for each class)
qda1 <- qda(f, data = train)

# 4. Naive Bayes (assumes predictors are independent within each class)
nb1 <- naiveBayes(f, data = train)

# 5. KNN. Predictors are standardised (using training means and SDs) because KNN
# uses distances. The number of neighbours k is chosen by leave-one-out
# cross-validation on the TRAINING data only, so the test year is never used.
preds <- c("temperature_2m_max", "temperature_2m_min",
           "wind_speed_10m_max", "shortwave_radiation_sum")
train_x <- scale(train[, preds])
test_x  <- scale(test[, preds],
                 center = attr(train_x, "scaled:center"),
                 scale  = attr(train_x, "scaled:scale"))
ks  <- seq(1, 61, by = 2)
cv_acc <- sapply(ks, function(k) mean(knn.cv(train_x, train$rain, k = k) == train$rain))
k_best <- ks[which.max(cv_acc)]
k_best
```

```
## [1] 13
```

```
plot(ks, cv_acc, type = "b", xlab = "k (number of neighbours)",
     ylab = "Leave-one-out accuracy (training data)",
     main = "Choosing k for KNN")
abline(v = k_best, lty = 2)
```



Interpretation. In the logistic regression, minimum temperature (coefficient 0.54), maximum temperature (-0.28) and radiation (-0.15) are all highly significant ($p < 0.001$). Wind speed is not significant ($p = 0.99$), so wind speed tells us almost nothing about rain once the other variables are known. The signs make physical sense. A day is more likely to be a rain day when the night is warm, the day is not so hot and there is less sunshine, which is what cloudy conditions look like. In other words, rain days in Parramatta have a small gap between the daily maximum and minimum, because cloud cover cools the day and keeps the night warm. The LDA group means agree with this. Rain days average a minimum of 14.7 °C against 11.6 °C on dry days, and radiation of 12.9 against 17.9 MJ/m². The best number of neighbours for KNN is $k = 13$. Small k values follow noise in the training data, and very large k values smooth away real structure.

Conclusion for Parramatta

Rain days in Parramatta are not random. Compared with dry days they tend to have warmer nights, a smaller gap between the daily maximum and minimum, and less incoming solar radiation, which is the signature of cloudy conditions. For these four variables the dry/rain boundary is close to linear, because the simpler logistic regression and LDA fit as well as or better than the more flexible QDA and KNN (Part 4); Parramatta's rain is therefore described mainly by how warm and how sunny the day was, and not by its maximum temperature alone.

Limitations and further analysis. (1) Every predictor is measured on the same day as the rain, so these models describe rainy conditions rather than forecast rain. (2) Wind speed was not significant, and the main drivers of Parramatta rain, such as humidity, air pressure, wind direction and the previous day's rain, are not in this dataset; adding them should improve the classifiers. (3) The 1 mm threshold for a rain day is a choice, and a different threshold would change the class balance and the results. (4) These classifiers are fitted on 2021 to 2024 only; Part 4 tests how well they generalise to the unseen 2025 data.

Part 4: Classification Validation

Aim

The aim is to test the five classifiers on the 2025 data they have never seen, using confusion matrices, accuracy, sensitivity and specificity. Here "positive" means a rain day. **Sensitivity** is the share of real rain days that the classifier correctly calls rain. **Specificity** is the share of real dry days that it correctly calls dry. All classifiers use the usual 0.5 probability cut-off.

Confusion matrices and performance

```
# Predicted classes on the held-out 2025 data
pred_glm <- factor(ifelse(predict(glm1, test, type = "response") > 0.5, "Rain", "Dry"),
                   levels = c("Dry", "Rain"))
pred_lda <- predict(lda1, test)$class
pred_qda <- predict(qda1, test)$class
pred_nb  <- predict(nb1, test)
pred_knn <- knn(train_x, test_x, train$rain, k = k_best)

preds_list <- list(Logistic = pred_glm, LDA = pred_lda, QDA = pred_qda,
                   NaiveBayes = pred_nb, KNN = pred_knn)

# Confusion matrix for each classifier (rows = predicted, columns = actual)
cms <- lapply(preds_list, function(p) table(Predicted = p, Actual = test$rain))
cms
```

```
## $Logistic
##          Actual
## Predicted Dry Rain
##      Dry  201   38
##      Rain  38   88
##
## $LDA
##          Actual
## Predicted Dry Rain
##      Dry  207   40
##      Rain  32   86
##
## $QDA
##          Actual
## Predicted Dry Rain
##      Dry  202   45
##      Rain  37   81
##
## $NaiveBayes
##          Actual
## Predicted Dry Rain
##      Dry  222   63
##      Rain  17   63
##
## $KNN
```

```
##           Actual
## Predicted Dry Rain
##      Dry  202   54
##      Rain  37   72
```

```r
# Accuracy, sensitivity (rain days found) and specificity (dry days found)
perf <- t(sapply(cms, function(cm) c(
  Accuracy    = sum(diag(cm)) / sum(cm),
  Sensitivity = cm["Rain", "Rain"] / sum(cm[, "Rain"]),
  Specificity = cm["Dry", "Dry"]   / sum(cm[, "Dry"]))))
knitr::kable(round(perf, 3),
             caption = "Performance on the 2025 test set (rain day = positive)")
```

Table 1: Performance on the 2025 test set (rain day = positive)

	Accuracy	Sensitivity	Specificity
Logistic	0.792	0.698	0.841
LDA	0.803	0.683	0.866
QDA	0.775	0.643	0.845
NaiveBayes	0.781	0.500	0.929
KNN	0.751	0.571	0.845

Which classifier is best, and why

By accuracy, the best classifier is **LDA** with 80.3% correct, compared with the 65.5% baseline of always predicting "Dry". Logistic regression (79.2%) is very close behind. With only 365 test days, a one percentage point difference is about 4 days, so the top two classifiers cannot really be separated and I would call them equally good. All five classifiers beat the baseline. Naive Bayes (78.1%) and QDA (77.5%) are a little behind, and KNN (75.1%) is the weakest.

A plausible explanation, based on the structure of the data:

1. **The boundary is close to linear.** The two linear methods did best, and the more flexible QDA and KNN did not improve on them. The difference between the classes is mainly a shift in the means (rain days have warmer nights and less radiation), and the extra flexibility of QDA and KNN only adds variance. The covariances are not identical in the two classes (for example the variance of maximum temperature is 30 on dry days against 20 on rain days), but with only 502 rain days in the training data QDA has to estimate many more parameters than LDA and does not gain from it.
2. **The predictors are correlated, which hurts naive Bayes.** Maximum temperature, minimum temperature and radiation are strongly correlated with each other (r about 0.7 for TempMax with each of TempMin and Radiation, see Part 1). Naive Bayes assumes they are independent given the class, so it is working from a wrong assumption. Its sensitivity of 0.5 is the lowest of all five, so it misses half of the rain days, although it has the highest specificity (0.93).
3. **KNN struggles with overlapping classes and a modest sample.** The two classes overlap a lot, because plenty of cloudy days do not produce more than 1 mm of rain. Distance-based voting is then noisy. Training on 1461 days is also small for a method that needs many neighbours around every point.
4. **Imbalance and cut-off.** All classifiers find dry days (specificity 0.84 to 0.93) more easily than rain days (sensitivity 0.5 to 0.7), because there are about twice as many dry days and 0.5 favours the larger class. If catching rain days mattered more than avoiding false alarms, the cut-off should be lowered below 0.5.

Conclusion for Parramatta

Rain days in Parramatta are recognisable from warm nights, cooler days and low solar radiation, and a simple linear classifier such as LDA or logistic regression identifies about 80% of days correctly using only these four same-day measurements. That is a clear improvement on the baseline but still leaves roughly one rain day in three undetected, so these four variables describe rainy weather rather than fully explain it.

Limitations and further analysis. (1) The test set is one year (365 days), so the accuracy figures are uncertain, and Part 5 uses cross-validation to get a more reliable estimate. (2) All predictors are measured on the same day as the rain, so this is a description of rainy conditions and not a forecast. (3) Useful predictors such as humidity, air pressure and wind direction are not in the dataset. Adding them, or the previous day's rain, should help a lot. (4) The 1 mm threshold is arbitrary, and results may change with a different threshold. (5) A cut-off tuned with ROC curves, and a check of whether a non-linear model such as a GAM does better, would be sensible next steps.

Part 5: Resampling Methods and Error Estimation

Aim

The 2025 test set in Part 4 is only 365 days, so its error is a noisy estimate of how the models would do on new data. The aim here is to get more reliable test-error estimates with cross-validation for two models: the **final regression model from Part 2** (Model 2, predicting daily maximum temperature) and the **logistic regression from Part 3** (predicting rain days). Here the models are refitted on all 1826 days and tested on the days held out in each fold.

Choice of resampling method

I use **10-fold cross-validation**. The days are randomly split into 10 groups. Each group in turn is held out, the model is fitted to the other nine, and the held-out group is predicted. Every day is predicted exactly once, by a model that never saw it. I chose 10 folds because it is a good compromise between bias and variance. Each model is trained on 90% of the data, so its error is close to the error of a model trained on everything (leave-one-out is slightly less biased, but is more expensive and its estimates are highly correlated with each other). The simple validation set approach is what Part 4 did, and it is the noisy method I am trying to improve on. Repeating the 10-fold split with different random seeds shows how stable the result is.

Weather on neighbouring days is similar (the residuals in Part 2 had a lag-1 autocorrelation of 0.27), so random folds put near-copies of each test day in the training set, which makes the error look a little too good. As a stricter check I also use **leave-one-year-out cross-validation**: each of the five years is predicted by a model fitted to the other four years.

```
# Error estimates by cross-validation for:
#   (a) the final regression model (m2): root mean squared error (RMSE) in degrees C
#   (b) the logistic regression for rain days (formula f): misclassification rate
set.seed(2026)                          # makes the random folds reproducible
n <- nrow(weather)

# Predictions for every day, each made by a model that did not see that day
cv_reg <- function(folds) {
  p <- numeric(n)
  for (k in sort(unique(folds))) {
    fit <- lm(formula(m2), data = weather[folds != k, ])
    p[folds == k] <- predict(fit, weather[folds == k, ])
  }
  p
}
cv_cls <- function(folds) {
  p <- character(n)
  for (k in sort(unique(folds))) {
    fit <- glm(f, data = weather[folds != k, ], family = binomial)
    p[folds == k] <- ifelse(predict(fit, weather[folds == k, ], type = "response") > 0.5,
                            "Rain", "Dry")
  }
  p
}
rmse <- function(p) sqrt(mean((weather$temperature_2m_max - p)^2))
miscl <- function(p) mean(p != weather$rain)
```

```r
folds10 <- sample(rep(1:10, length.out = n))    # random 10 folds
folds_year <- weather$year                       # one fold per year
```

```r
# Training errors: model fitted and tested on the same data
glm_all <- glm(f, data = weather, family = binomial)
train_rmse <- sqrt(mean(resid(m2)^2))
train_err  <- mean(ifelse(fitted(glm_all) > 0.5, "Rain", "Dry") != weather$rain)

# Cross-validated errors
cv10_rmse <- rmse(cv_reg(folds10));  cv10_err <- miscl(cv_cls(folds10))
loyo_rmse <- rmse(cv_reg(folds_year)); loyo_err <- miscl(cv_cls(folds_year))

# Repeat 10-fold CV 20 times with different random splits to see how stable it is
rep_rmse <- sapply(1:20, function(i) rmse(cv_reg(sample(rep(1:10, length.out = n)))))
rep_err  <- sapply(1:20, function(i) miscl(cv_cls(sample(rep(1:10, length.out = n)))))

results <- data.frame(
  Estimate = c("Training error", "10-fold CV", "Leave-one-year-out CV"),
  Regression_RMSE_C = c(train_rmse, cv10_rmse, loyo_rmse),
  Logistic_error_rate = c(train_err, cv10_err, loyo_err))
knitr::kable(
  results, digits = 3,
  caption = "Training and cross-validated error for the final models")
```

Table 2: Training and cross-validated error for the final models

Estimate	Regression_RMSE_C	Logistic_error_rate
Training error	2.490	0.182
10-fold CV	2.504	0.182
Leave-one-year-out CV	2.552	0.187

```r
c(regression_rmse_sd_over_20_repeats = sd(rep_rmse),
  logistic_error_sd_over_20_repeats = sd(rep_err))
```

```
## regression_rmse_sd_over_20_repeats  logistic_error_sd_over_20_repeats
##                       0.0018124483                       0.0009731909
```

```r
# Leave-one-year-out RMSE for each year separately
p_year <- cv_reg(folds_year)
sapply(sort(unique(weather$year)), function(y) {
  i <- weather$year == y
  sqrt(mean((weather$temperature_2m_max[i] - p_year[i])^2))
})
```

```
## [1] 2.353441 2.215558 2.760420 2.546378 2.830722
```

Interpretation. For the regression model, the 10-fold CV estimate of test error is an RMSE of 2.5 °C, which is almost identical to the training RMSE of 2.49 °C. For the rain-day logistic regression, the 10-fold CV misclassification rate is 18.2% against a training error of 18.2%, which is again nearly the same (so the CV accuracy is about 81.8%). Repeating the 10-fold split 20 times changes the regression RMSE by only

about 0.002 °C and the error rate by about 0.1 percentage points, so the choice of random split does not matter. The stricter leave-one-year-out estimates are a little worse (2.55 °C and 18.7%). That is the expected direction, because random folds are a bit optimistic when neighbouring days are alike. It also varies by year. The regression RMSE ranges from 2.22 °C (2022) to 2.83 °C (2025). The held-out year 2025 in Parts 3 and 4 gave a logistic error of 20.8%, higher than the CV estimate of 18.2%, probably because 2025 happened to be a harder year to predict (its regression error was also the highest of the five years). This shows why one test set can mislead, and why the cross-validated figures are the ones I would quote.

Conclusion for Parramatta

Cross-validation gives a more trustworthy picture of how well these models would work on new days in Parramatta than the single 2025 split of Parts 3 and 4. The regression that predicts the daily maximum temperature from same-day conditions has a typical error of about 2.5 °C, and the rain-day logistic regression misclassifies about 18.2% of days. Both figures are close to the training errors, so there is little sign of overfitting, but the stricter leave-one-year-out check is a little worse (2.55 °C), which shows that the particular year held out matters for a place with Parramatta's year-to-year variability.

Limitations and further analysis. (1) Random folds ignore the time order of the data, so the 10-fold figures are mildly optimistic where consecutive days are alike, and leave-one-year-out corrects for this but rests on only five years. (2) Both models still use same-day measurements, so the errors describe Parramatta's weather rather than a forecast. (3) A blocked or time-series cross-validation scheme, and a longer record, would give firmer error estimates.

Part 6: Coefficient Standard Error

Aim

The aim is to estimate the standard error of one coefficient from the final Part 2 regression (Model 2) with the **bootstrap**, and to compare it with the standard error reported by `lm()`. I chose the coefficient for **maximum wind speed** (`wind_speed_10m_max`), because it is not part of the interaction term, so it has a single simple interpretation: the change in maximum temperature per extra km/h of wind.

Bootstrap standard error

The bootstrap draws 2000 new data sets of the same size by sampling days **with replacement** from the original data, refits Model 2 to each, and records the wind coefficient. The standard deviation of those 2000 coefficients is the bootstrap standard error.

```
set.seed(2026)
B <- 2000
coef_name <- "wind_speed_10m_max"

# Standard bootstrap: resample individual days with replacement
boot_iid <- replicate(B, {
  i <- sample(n, n, replace = TRUE)
  coef(lm(formula(m2), data = weather[i, ]))[coef_name]
})

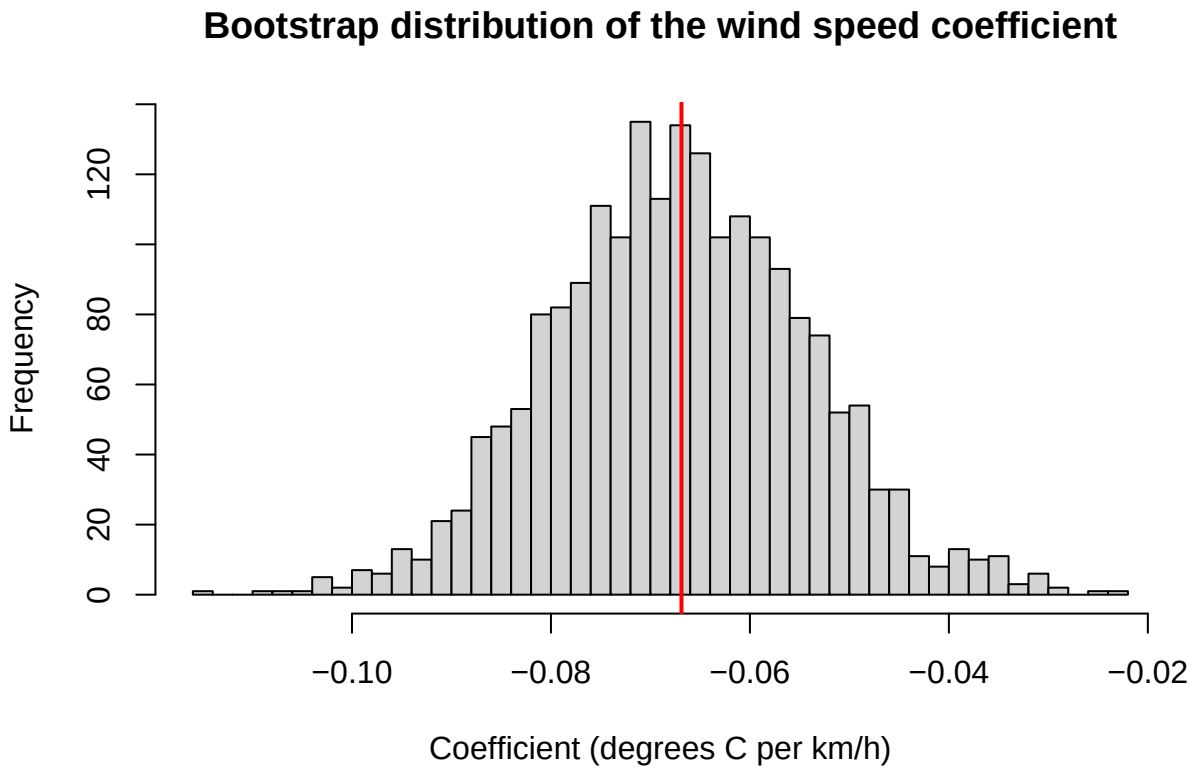
# Block bootstrap: resample blocks of 30 consecutive days, to respect the
# day-to-day dependence found in Part 2 (autocorrelation of residuals)
L <- 30
n_blocks <- ceiling(n / L)
boot_block <- replicate(B, {
  starts <- sample(1:(n - L + 1), n_blocks, replace = TRUE)
  i <- unlist(lapply(starts, function(s) s:(s + L - 1)))[1:n]
  coef(lm(formula(m2), data = weather[i, ]))[coef_name]
})

# Compare the three standard errors and 95% percentile intervals
se_lm <- summary(m2)$coefficients[coef_name, "Std. Error"]
comparison <- data.frame(
  Method = c("lm() formula", "Bootstrap (resample days)",
             "Block bootstrap (30-day blocks)"),
  Estimate = coef(m2)[coef_name],
  Std_Error = c(se_lm, sd(boot_iid), sd(boot_block)),
  CI_lower = c(confint(m2)[coef_name, 1], quantile(boot_iid, 0.025),
               quantile(boot_block, 0.025)),
  CI_upper = c(confint(m2)[coef_name, 2], quantile(boot_iid, 0.975),
               quantile(boot_block, 0.975)),
  row.names = NULL)
knitr::kable(
  comparison, digits = 4,
  caption = "Standard error of the wind speed coefficient")
```

Table 3: Standard error of the wind speed coefficient

Method	Estimate	Std_Error	CI_lower	CI_upper
lm() formula	-0.0669	0.0120	-0.0903	-0.0434
Bootstrap (resample days)	-0.0669	0.0129	-0.0917	-0.0412
Block bootstrap (30-day blocks)	-0.0669	0.0163	-0.0962	-0.0330

```
# Histogram of the bootstrap coefficients
hist(boot_iid, breaks = 40,
     main = "Bootstrap distribution of the wind speed coefficient",
     xlab = "Coefficient (degrees C per km/h)")
abline(v = coef(m2)[coef_name], col = "red", lwd = 2)
```



Interpretation. The `lm()` standard error is 0.012, and the standard bootstrap gives 0.0129, which is 8% larger. The two are close, which supports using the `lm()` formulas for this coefficient, but the bootstrap is a little bigger. This is consistent with the Part 2 diagnostics, where the residuals had unequal variance and heavy tails. `lm()` assumes constant variance and normal errors, and the bootstrap does not. The block bootstrap gives a larger standard error again (0.0163, 36% above `lm()`), because it keeps the day-to-day dependence in the data. This is what the lag-1 residual correlation of 0.27 in Part 2 suggested: the `lm()` standard errors are slightly too small because neighbouring days are not independent. Even so, the 95% interval from the block bootstrap (-0.096 to -0.033) stays below zero, so the conclusion that stronger wind goes with a cooler maximum temperature in Parramatta does not change.

Training error against cross-validated error: overfitting

The Part 5 results show almost no gap between training and cross-validated error. For the regression the training RMSE is 2.49 °C and the 10-fold CV RMSE is 2.504 °C, a difference of 0.014 °C. For the logistic regression the training error is 18.2% and the CV error is 18.2%. Overfitting would show up as a training error that is much lower than the CV error. It is absent here because the models are simple (8 coefficients for the regression and 5 for the logistic model) and are fitted to 1826 days, so there is little room to fit noise. The same reasoning explains why the more flexible QDA and KNN in Part 4 did not beat the linear methods. The one sign of a gap is the stricter leave-one-year-out result (2.55 °C), which is a little worse than the random-fold estimate. That is not overfitting to noise. It reflects that individual years differ from each other, and a model trained on other years carries that extra uncertainty.

Conclusion for Parramatta

The models for Parramatta's weather are about as good on new days as they are on the days they were fitted to. A typical error in predicting the daily maximum temperature is about 2.5 °C, and about 18% of days are misclassified as rain or dry, with somewhat larger errors for a whole unseen year. The standard errors from `lm()` are reasonable, but the bootstrap, and especially the block bootstrap, shows they are a little optimistic because weather on consecutive days is related.

Limitations and further analysis. (1) Random folds ignore the time order of the data, and leave-one-year-out is based on only five years, so the estimates are not precise. (2) The block length of 30 days was chosen by judgement, and other lengths would give somewhat different standard errors. (3) Only the wind coefficient was bootstrapped, and the other coefficients could behave differently. (4) A time-series model with the previous day's weather as predictors, and a longer record of Parramatta weather, would give firmer conclusions.

Part 7: Moving Beyond Linearity

Aim

The aim is to describe the seasonal pattern in Parramatta's daily maximum temperature as a function of **day of year** (1 January = 1, 31 December = 365 or 366). Part 1 suggested that a straight line would fit badly, and here I check that and fit four non-linear models with hyperparameters that I choose by hand: a polynomial regression, a natural (regression) spline, a smoothing spline and a GAM. Part 8 then chooses the flexibility properly with cross-validation.

Models with manually chosen flexibility

My manual choices are based on the shape of the seasonal curve in the scatterplot: one smooth hot season and one smooth cool season each year, with no sharp features. I would expect a few degrees of freedom to be enough.

- **Polynomial regression**, degree 4.
- **Natural cubic spline** (regression spline) with 6 degrees of freedom, so 5 knots placed at quantiles of day of year.
- **Smoothing spline** with 8 effective degrees of freedom.
- **GAM** with a single smooth term of day of year, a cubic regression spline with 8 basis functions (7 degrees of freedom) and no penalty.

```
library(splines)   # ns()
library(mgcv)      # gam()

y <- weather$temperature_2m_max
x <- weather$doy
n <- nrow(weather)

# Baseline: a straight line in day of year
lin <- lm(temperature_2m_max ~ doy, data = weather)

# 1. Polynomial regression, degree 4 (poly() uses orthogonal polynomials)
poly4 <- lm(temperature_2m_max ~ poly(doy, 4), data = weather)

# 2. Natural cubic spline with 6 degrees of freedom
ns6 <- lm(temperature_2m_max ~ ns(doy, df = 6), data = weather)

# 3. Smoothing spline with 8 effective degrees of freedom
ss8 <- smooth.spline(x, y, df = 8)

# 4. GAM with one smooth term for day of year (k = 8 basis functions, fixed df)
gam8 <- gam(temperature_2m_max ~ s(doy, k = 8, fx = TRUE), data = weather)

# Fitted values from each model
fits <- list(Linear = fitted(lin), Poly4 = fitted(poly4), NS6 = fitted(ns6),
             SmoothSpline8 = predict(ss8, x)$y, GAM8 = fitted(gam8))

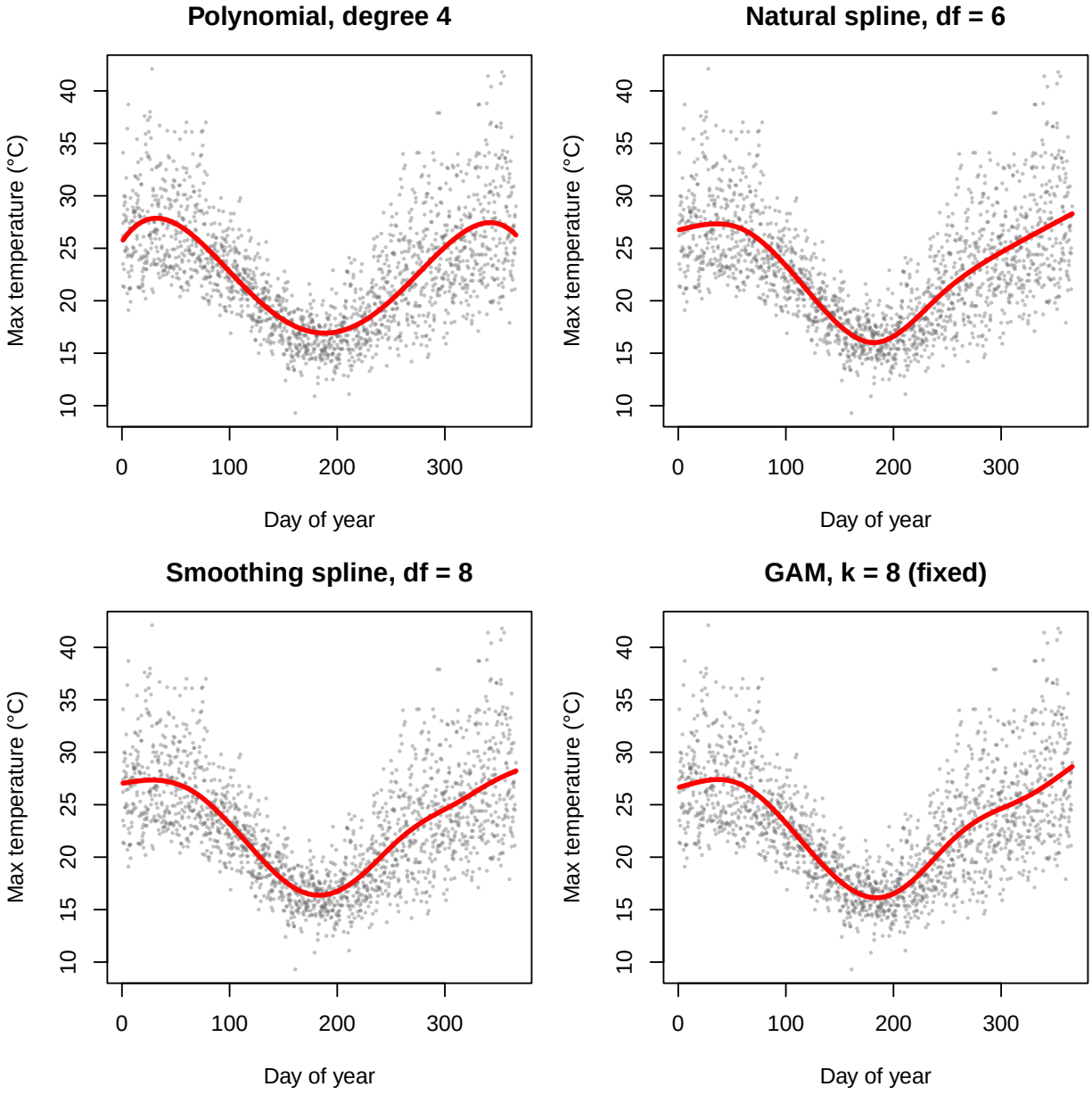
# Goodness-of-fit statistics
tss <- sum((y - mean(y))^2)
gof <- data.frame(
```

```
  Model = names(fits),
  R2    = sapply(fits, function(f) 1 - sum((y - f)^2) / tss),
  RMSE  = sapply(fits, function(f) sqrt(mean((y - f)^2))),
  AIC   = c(AIC(lin), AIC(poly4), AIC(ns6), NA, AIC(gam8)),
  row.names = NULL)
cap <- "In-sample fit of the manually chosen models (AIC not defined for smooth.spline)"
knitr::kable(gof, digits = 3, caption = cap)
```

Table 4: In-sample fit of the manually chosen models (AIC not defined for smooth.spline)

Model	R2	RMSE	AIC
Linear	0.003	5.406	11350.779
Poly4	0.528	3.720	9991.901
NS6	0.540	3.672	9948.679
SmoothSpline8	0.540	3.671	NA
GAM8	0.540	3.671	9949.096

```
# Data with each fitted curve, one panel per model
xs <- 1:366
curves <- list(
  "Polynomial, degree 4"      = predict(poly4, data.frame(doy = xs)),
  "Natural spline, df = 6"    = predict(ns6, data.frame(doy = xs)),
  "Smoothing spline, df = 8"  = predict(ss8, xs)$y,
  "GAM, k = 8 (fixed)"        = predict(gam8, data.frame(doy = xs)))
par(mfrow = c(2, 2), mar = c(4, 4, 3, 1))
for (nm in names(curves)) {
  plot(x, y, pch = 16, cex = 0.4, col = adjustcolor("grey40", 0.4),
       xlab = "Day of year", ylab = "Max temperature (°C)", main = nm)
  lines(xs, curves[[nm]], col = "red", lwd = 3)
}
```




```
par(mfrow = c(1, 1))
```

```
# Sequential F-tests: does each extra polynomial degree reduce the
# residual sum of squares?
an <- anova(lm(y ~ doy, data = weather),
            lm(y ~ poly(doy, 2), data = weather),
            lm(y ~ poly(doy, 3), data = weather),
            lm(y ~ poly(doy, 4), data = weather),
            lm(y ~ poly(doy, 5), data = weather),
            lm(y ~ poly(doy, 6), data = weather))
an
```

```
## Analysis of Variance Table
```

```
##
## Model 1: y ~ doy
## Model 2: y ~ poly(doy, 2)
## Model 3: y ~ poly(doy, 3)
## Model 4: y ~ poly(doy, 4)
## Model 5: y ~ poly(doy, 5)
## Model 6: y ~ poly(doy, 6)
##   Res.Df   RSS Df Sum of Sq        F    Pr(>F)
## 1   1824 53365
## 2   1823 29950  1   23415.9 1718.528 < 2.2e-16 ***
## 3   1822 29795  1     154.2   11.319 0.0007832 ***
## 4   1821 25272  1    4523.4  331.977 < 2.2e-16 ***
## 5   1820 25141  1     131.1    9.622 0.0019521 **
## 6   1819 24785  1     355.9   26.123  3.54e-07 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
# Residual lag-1 autocorrelation for the natural spline
res_ns <- resid(ns6)
cor(res_ns[-1], res_ns[-length(res_ns)])
```

```
## [1] 0.4931458
```

Interpretation. A straight line is useless here. It has an R-squared of 0.003, because the temperature rises and falls over the year. The four non-linear models are all much better and are almost indistinguishable. Their R-squared values are between 0.528 and 0.54 and their RMSE is about 3.7 °C. The smooth curves peak in summer and bottom out in winter, with no sign of kinks, so the extra flexibility of a small number of degrees of freedom is enough. In the polynomial F-tests the residual sum of squares drops from 53,365 (degree 1) to 29,950 (degree 2), changes little at degree 3 (29,795), falls again at degree 4 (25,272) and then gains only a little more. All the extra terms are statistically significant, but with 1826 days even small gains are significant, so I judge by size, and degree 4 captures nearly all the improvement. A modest R-squared of about 0.54 should not be read as a poor fit. It means that the average seasonal cycle explains about half of the day-to-day variation in Parramatta's maximum temperature, and the other half (RMSE of about 3.7 °C) is weather that sits on top of the seasons, such as heatwaves and cool changes. The residuals of the natural spline have a lag-1 autocorrelation of 0.49, so a hot day tends to be followed by another hot day. This is the same dependence that was found in Part 2.

Conclusion for Parramatta

The seasonal pattern in Parramatta's maximum temperature is clearly non-linear. A straight line in day of year explains almost none of the variation (R-squared 0.003), while a smooth seasonal curve explains about 0.54 of it. All four methods peak in summer and bottom out in winter and are almost indistinguishable from each other, which tells me that Parramatta's seasonality is a broad, smooth curve rather than anything with sharp features. The variation that remains (an RMSE of about 3.7 °C) is the day-to-day weather that sits on top of the seasons, such as heatwaves and cool changes.

Limitations and further analysis. (1) The flexibility of each model is chosen by hand from the shape of the curve, so these settings are subjective; Part 8 selects them properly with cross-validation. (2) The residuals are autocorrelated (0.49), so a hot day tends to follow a hot day and the usual independent-error assumptions are only approximate. (3) A smooth curve in day of year gives the typical temperature for a date, not a prediction for a particular day; adding the same-day predictors from Part 2 in a single GAM would be a natural extension.

Part 8: Validating the Non-linear Models

Aim

The aim is to choose the flexibility of each model (polynomial degree, spline degrees of freedom, smoothing spline degrees of freedom, GAM basis size) using cross-validation, to plot the chosen curves together, and to recommend one model for describing Parramatta's seasonal temperature pattern. I also add a fifth model that I think suits this problem: a GAM with a **cyclic** cubic spline, which forces the curve to join up smoothly between 31 December and 1 January.

Choosing flexibility by cross-validation

I use random 10-fold cross-validation (the same method as Part 5) to compare the models over a range of flexibilities, and leave-one-year-out cross-validation as a stricter check, because neighbouring days are similar and random folds are slightly optimistic. The rule for choosing is: take the **least flexible** setting whose 10-fold CV error is within 0.01 °C of the best. That is a small tolerance (about 0.3% of the error), and it prefers simpler models when the differences are too small to matter.

```
set.seed(2026)
folds10 <- sample(rep(1:10, length.out = n))    # random folds
folds_year <- weather$year                       # one fold per year

# Cross-validated RMSE for a model-fitting function fp(train, test) -> predictions
cv_rmse <- function(fp, folds) {
  p <- numeric(n)
  for (k in sort(unique(folds))) {
    p[folds == k] <- fp(weather[folds != k, ], weather[folds == k, ])
  }
  sqrt(mean((y - p)^2))
}

# Model families: each returns a function of (train, test)
poly_fp <- function(d) function(tr, te)
  predict(lm(temperature_2m_max ~ poly(doy, d), data = tr), te)
ns_fp <- function(d) function(tr, te)
  predict(lm(temperature_2m_max ~ ns(doy, df = d), data = tr), te)
ss_fp <- function(d) function(tr, te)
  predict(smooth.spline(tr$doy, tr$temperature_2m_max, df = d), te$doy)$y
gam_fp <- function(k) function(tr, te)
  predict(gam(temperature_2m_max ~ s(doy, k = k, fx = TRUE), data = tr), te)
cc_fp <- function(k) function(tr, te)
  predict(gam(temperature_2m_max ~ s(doy, k = k, bs = "cc"), data = tr,
              knots = list(doy = c(0.5, 366.5))), te)

# Grid of flexibilities for each family
grids <- list(Polynomial = 1:10, NaturalSpline = 2:14,
              SmoothingSpline = c(3, 4, 5, 6, 8, 10, 12, 15, 20),
              GAM_fixed = c(4, 5, 6, 7, 8, 10, 12), GAM_cyclic = c(4, 5, 6, 7, 8, 10))
fps <- list(Polynomial = poly_fp, NaturalSpline = ns_fp, SmoothingSpline = ss_fp,
            GAM_fixed = gam_fp, GAM_cyclic = cc_fp)

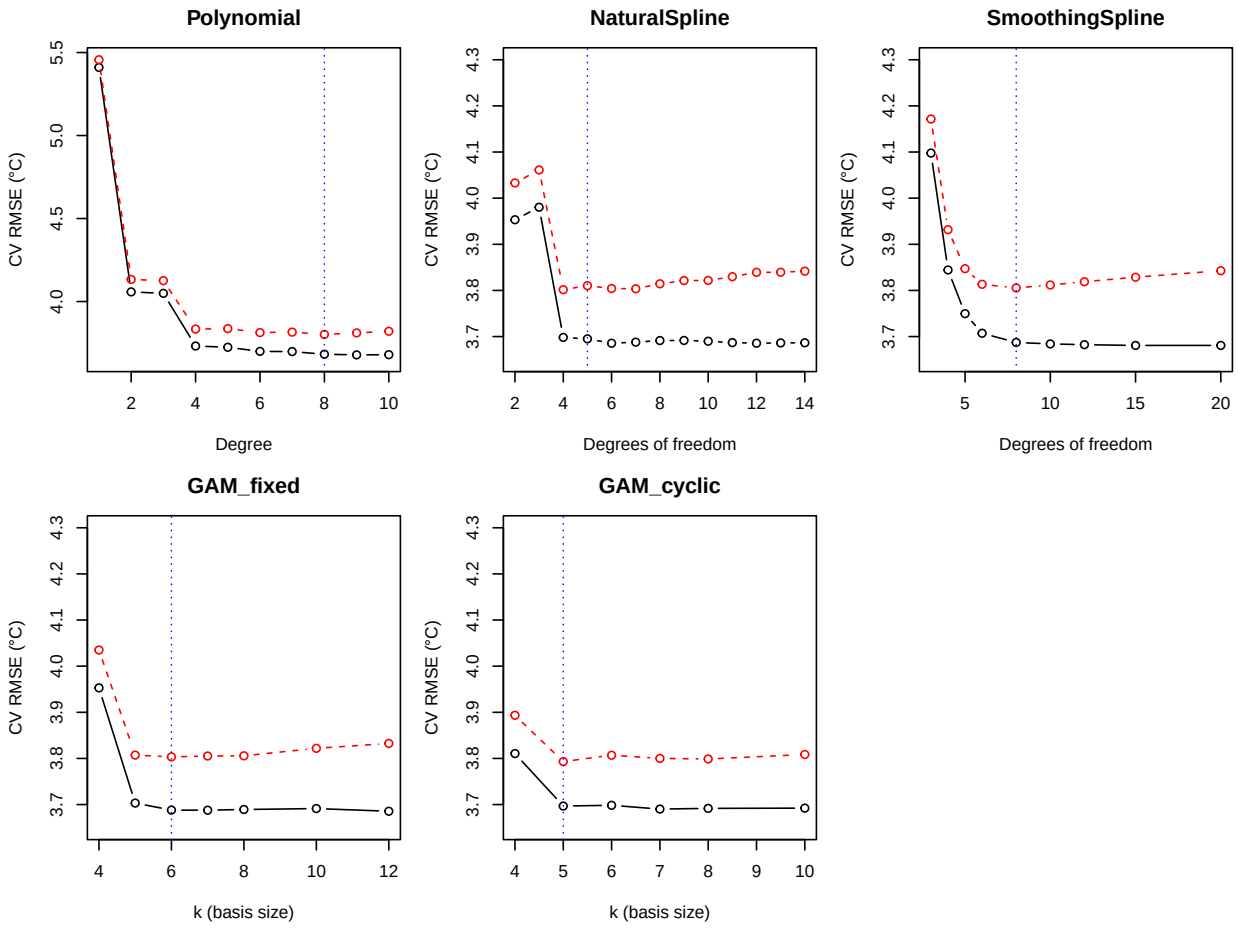
cv_tab <- lapply(names(grids), function(m) {
  data.frame(model = m, par = grids[[m]],
```

```
            cv10 = sapply(grids[[m]], function(g) cv_rmse(fps[[m]](g), folds10)),
            loyo = sapply(grids[[m]], function(g) cv_rmse(fps[[m]](g), folds_year)))
})
names(cv_tab) <- names(grids)

# Selection rule: least flexible setting within 0.01 C of the best 10-fold CV error
chosen <- sapply(cv_tab, function(d) min(d$par[d$cv10 <= min(d$cv10) + 0.01]))
chosen
```

```
##      Polynomial   NaturalSpline SmoothingSpline       GAM_fixed      GAM_cyclic
##               8               5               8               6               5
```

```
# CV error against flexibility; solid = 10-fold, dashed = leave-one-year-out,
# vertical line = chosen setting
par(mfrow = c(2, 3), mar = c(4, 4, 3, 1))
xlabs <- c(Polynomial = "Degree", NaturalSpline = "Degrees of freedom",
           SmoothingSpline = "Degrees of freedom", GAM_fixed = "k (basis size)",
           GAM_cyclic = "k (basis size)")
for (m in names(cv_tab)) {
  d <- cv_tab[[m]]
  plot(d$par, d$cv10, type = "b", ylim = range(3.65, 4.3, d$cv10, d$loyo),
       xlab = xlabs[m], ylab = "CV RMSE (°C)", main = m)
  lines(d$par, d$loyo, type = "b", lty = 2, col = "red")
  abline(v = chosen[m], col = "blue", lty = 3)
}
par(mfrow = c(1, 1))
```




```
# Chosen models: final fits on all data, with CV errors at the chosen setting
poly_f <- lm(temperature_2m_max ~ poly(doy, chosen["Polynomial"]), data = weather)
ns_f   <- lm(temperature_2m_max ~ ns(doy, df = chosen["NaturalSpline"]), data = weather)
ss_f   <- smooth.spline(x, y, df = chosen["SmoothingSpline"])
gam_f  <- gam(temperature_2m_max ~ s(doy, k = chosen["GAM_fixed"], fx = TRUE),
              data = weather)
cc_f   <- gam(temperature_2m_max ~ s(doy, k = chosen["GAM_cyclic"], bs = "cc"),
              data = weather, knots = list(doy = c(0.5, 366.5)))

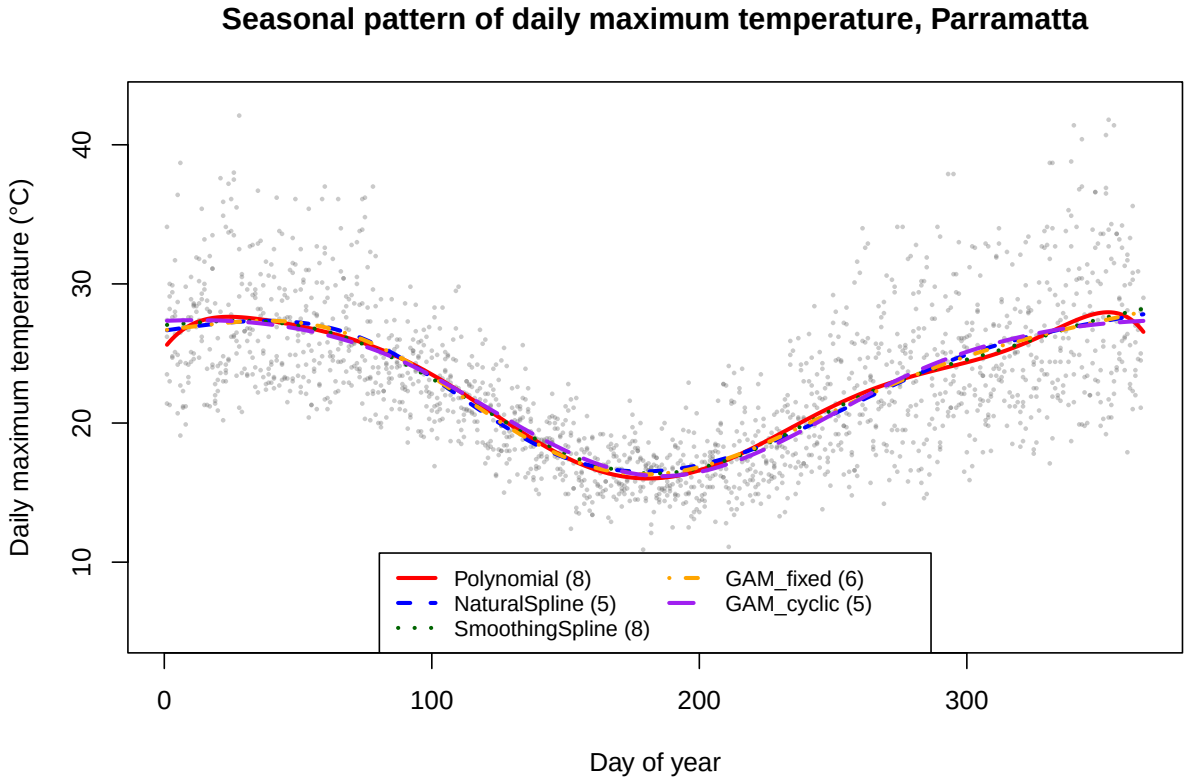
final_fits <- list(Polynomial = fitted(poly_f), NaturalSpline = fitted(ns_f),
                   SmoothingSpline = predict(ss_f, x)$y, GAM_fixed = fitted(gam_f),
                   GAM_cyclic = fitted(cc_f))
pick <- function(m, col) cv_tab[[m]][cv_tab[[m]]$par == chosen[m], col]
summary_tab <- data.frame(
  Model = names(chosen),
  Chosen = chosen,
  CV10_RMSE  = sapply(names(chosen), pick, col = "cv10"),
  LOYO_RMSE  = sapply(names(chosen), pick, col = "loyo"),
  Train_R2   = sapply(final_fits, function(f) 1 - sum((y - f)^2) / tss),
  row.names = NULL)
knitr::kable(
  summary_tab, digits = 3,
  caption = "Chosen flexibility and cross-validated error for each model")
```

Table 5: Chosen flexibility and cross-validated error for each model

Model	Chosen	CV10_RMSE	LOYO_RMSE	Train_R2
Polynomial	8	3.683	3.802	0.542
NaturalSpline	5	3.695	3.810	0.537
SmoothingSpline	8	3.687	3.805	0.540
GAM_fixed	6	3.688	3.803	0.539
GAM_cyclic	5	3.697	3.793	0.536

Comparing the fitted curves

```r
# All chosen curves on one plot, over the data
final_curves <- list(
  Polynomial      = predict(poly_f, data.frame(doy = xs)),
  NaturalSpline   = predict(ns_f, data.frame(doy = xs)),
  SmoothingSpline = predict(ss_f, xs)$y,
  GAM_fixed       = predict(gam_f, data.frame(doy = xs)),
  GAM_cyclic      = predict(cc_f, data.frame(doy = xs)))
cols <- c("red", "blue", "darkgreen", "orange", "purple")
plot(x, y, pch = 16, cex = 0.4, col = adjustcolor("grey40", 0.35), ylim = c(5, 43),
     xlab = "Day of year", ylab = "Daily maximum temperature (°C)",
     main = "Seasonal pattern of daily maximum temperature, Parramatta")
for (i in seq_along(final_curves))
  lines(xs, final_curves[[i]], col = cols[i], lwd = 2.5, lty = i)
legend("bottom", legend = paste0(names(chosen), " (", chosen, ")"),
       col = cols, lty = 1:5, lwd = 2.5, cex = 0.8, ncol = 2, bg = "white")
```




```
# Fitted value at 1 January and 31 December (leap-year day 366): should be about equal
edges <- sapply(final_curves, function(cv) cv[c(1, 366)])
rownames(edges) <- c("1 Jan", "31 Dec")
round(t(edges), 2)
```

```
##                 1 Jan 31 Dec
## Polynomial      25.63  26.55
## NaturalSpline   26.67  27.82
## SmoothingSpline 27.06  28.22
## GAM_fixed       26.70  28.01
## GAM_cyclic      27.35  27.34
```

```
# Where the cyclic curve peaks and bottoms out
c(hottest_day = which.max(final_curves$GAM_cyclic),
  coolest_day = which.min(final_curves$GAM_cyclic))
```

```
##  hottest_day.12 coolest_day.186
##              12             186
```

Interpretation. The CV curves show the same pattern for all five families. The error falls steeply while the models are too rigid (for example polynomial degree 1 to 3, or only 2 to 3 degrees of freedom), then flattens out around 3.68 °C. After that, more flexibility gives no real improvement, and for the splines and GAMs the leave-one-year-out error actually gets a little worse as flexibility increases, which is a sign of the fits starting to follow noise. The chosen settings are polynomial degree 8, natural spline with 5 degrees of

freedom, smoothing spline with 8, GAM with k = 6 and cyclic GAM with k = 5. As a cross-check, the default automatic choice of `smooth.spline()` gives about 15 degrees of freedom, which is in the flat part of the curve and so agrees with my grid. The differences in CV error between the five chosen models are tiny (10-fold errors from 3.683 to 3.697 °C), so by prediction accuracy alone they are effectively tied. The polynomial needed degree 8 to get there, which is a lot of coefficients for such a simple seasonal shape, while the splines needed only 5 to 8 degrees of freedom.

The five curves in the plot lie almost on top of each other through the middle of the year, with the hottest part of the year around day 12 (12 January) and the coolest around day 186 (5 July) in the cyclic fit. They differ at the two ends of the year. Temperature on 31 December should be about the same as on 1 January, because the two days are next to each other in the calendar. The cyclic spline guarantees this (its two ends differ by 0.01 °C), but the natural spline does not (a gap of 1.15 °C), and the other non-cyclic models have gaps of 0.93 °C (polynomial), 1.16 °C (smoothing spline) and 1.31 °C (fixed GAM) at the edges, where a model has less data to rely on. These gaps are about 1 °C, which is small next to the day-to-day noise of about 3.7 °C, so the practical effect is modest, but it is a real difference in how sensible the curves are.

Recommendation for Parramatta

I recommend the **cyclic GAM with k = 5**. Its cross-validated error is essentially as good as that of any other model (10-fold 3.697 °C, which is within 0.014 °C of the best, and leave-one-year-out 3.793 °C, the lowest of the five on the stricter check), and it has only 4 degrees of freedom, so it is the simplest. It is also the only one that is guaranteed to give the same temperature at the end and the start of the year, which matches how a calendar works, and a GAM can be extended to more predictors. If a simpler tool is needed, the natural spline with 5 degrees of freedom is a good alternative, but with a small mismatch at the year boundary. I would not use the high-degree polynomial.

For Parramatta, the seasonal curve shows a hot summer and a mild winter with a smooth transition between them. Even the best seasonal curve leaves a residual RMSE of about 3.7 °C, much larger than the 2.5 °C of the Part 2 regression, which used same-day TempMin and Radiation. So the time of year tells us what is typical for a date, but a good prediction of an individual day also needs that day's actual conditions.

Limitations and further analysis. (1) Only five years of data are used, so the curve is a five-year average and the year-to-year differences (such as hot summers) are not captured. (2) The residuals are autocorrelated (about 0.5), so the CV errors are slightly optimistic and standard errors for the curves would be too small. (3) The grid of flexibilities and the 0.01 °C tolerance in the selection rule are my own choices, and a different rule would pick slightly different settings, though the CV errors show it would not matter much. (4) The seasonal models use day of year only. Adding a year trend or the Part 2 predictors in a GAM, or fitting an additive model with an autoregressive error, would be a natural next step. (5) A longer record would show whether the seasonal shape is stable or shifting with climate.